lifetimes are important when resolving services outside the context of an HTTP request.

We'll start by revisiting the `EmailSender` service from chapter 8 to see how you could register the dependency graph in your DI container.

9.1 Registering custom services with the DI container

In this section you'll learn how to register your own services with the DI container. We'll explore the difference between a service and an implementation, and learn how to register the `EmailSender` hierarchy introduced in chapter 8.

In chapter 8 I described a system for sending emails when a new user registers in your application. Initially, the minimal API endpoint handler `RegisterUser` created an instance of `EmailSender` manually, using code similar to the following listing (which you saw in chapter 8).

Listing 9.1 Creating an `EmailSender` instance without dependency injection

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);    ❶

app.Run();

string RegisterUser(string username)
{
    IEmailSender emailSender = new EmailSender(      ❷
        new MessageFactory(),                        ❸
        new NetworkClient(                           ❹
```